Module 2 — Data Manipulation Practice

1 coding question, ~10-15 min, ~15-20 lines of code.

Key facts:

- This is general Python — strings, arrays, basic data structures. No ML.
- `numpy` and `sortedcontainers` are auto-imported. Standard library available.
- Expect nested loops or simple data structure usage. No algorithmic tricks required.
- Problems are wrapped in business narratives — read carefully.

Work each problem from scratch, then check against the solution.

Problem 1 — Triple Letters (from the official PDF)

Given a string sentence of English words separated by whitespace, count the number of words that contain a **triple duplicate letter** — i.e., the same letter appearing at least 3 times within the word. Case-insensitive.

Example:

```
sentence = "Dooddle moodle Pepper unsuccessfully"
output = 3
```

- "Dooddle": 'd' appears 3 times ✓
- "moodle": no letter 3 times ✗
- "Pepper": 'p' appears 3 times ✓
- "unsuccessfully": 'u' appears 3 times, 's' appears 3 times ✓

In [1]:
```
from collections import OrderedDict
import numpy as np
```

In [3]:
```
from sortedcontainers import SortedList, SortedDict, SortedSet
```

In [10]:
```
print(ord("A")-ord("a"))
```

```
-32
```

In [19]:
```
ord("a")
```

Out[19]:
```
97
```

In [13]:
```
print(ord("c")-ord("C"))
```

```
32
```

In [39]:
```
ord('U') < ord('a')
```

Out[39]:
```
True
```

In [58]:
```
max( set(sub_s) , key = sub_s.count )
```

Out[58]:
```
3
```

In [53]:
```
def lower_casify(curr_string):
    curr_string_set = set(curr_string)
    new_string = curr_string[:]
    for char in curr_string_set:
        old_ord_char = ord(char)
        if old_ord_char < ord("a"):
            new_ord_char = old_ord_char + 32
            new_string = curr_string.replace(
                chr(old_ord_char), chr(new_ord_char))
        curr_string = new_string

    return new_string
```

In [62]:
```
sum([True, True, False])
```

Out[62]:
```
2
```

In [63]:
```
def lower_casify(curr_string):
    curr_string_set = set(curr_string)
    new_string = curr_string[:]
    for char in curr_string_set:
        old_ord_char = ord(char)
        if old_ord_char < ord("a"):
            new_ord_char = old_ord_char + 32
            new_string = curr_string.replace(
                chr(old_ord_char), chr(new_ord_char))
        curr_string = new_string

    return new_string

def solution(sentence):
    # implement this
    # separate words
    ndups_max_per_word = []
    for sub_s in sentence.split():
        sub_s = lower_casify(sub_s)
        nreps = sub_s.count(
            max( set(sub_s) , key = sub_s.count ))
        ndups_max_per_word.append(nreps)

    return sum([i>=3 for i in ndups_max_per_word ])

# Test
assert solution("Dooddle moodle Pepper unsuccessfully") == 3
```

```
assert solution("hello world") == 0
assert solution("aaa bbb") == 2
print("All tests passed")
```

```
All tests passed
```

▶ Solution

```
def solution(sentence):
    count = 0
    for word in sentence.split():
        word = word.lower()
        if any(word.count(c) >= 3 for c in set(word)):
            count += 1
    return count
```

Problem 2 — Array Manipulation

Given an array of integers, create two new arrays:

- `evens` : all even numbers from the array, doubled
- `odds` : all odd numbers from the array, with 1 subtracted

Return `odds + evens` (odds array prepended to evens array).

Example:

```
arr = [1, 2, 3, 4, 5]
evens = [4, 8]   # 2*2, 4*2
odds = [0, 2, 4]  # 1-1, 3-1, 5-1
output = [0, 2, 4, 4, 8]
```

In [84]:
```
arr=np.array([5,4,3,2,1])
arr[ arr%2 == 1 ] -=1
```

In [85]:
```
import numpy as np
```

In [86]:
```
np.concatenate(arr, arr)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[86], line 1
----> 1 np.concatenate(arr, arr)

TypeError: only integer scalar arrays can be converted to a scalar index
```

In [88]:
```
def solution(arr):
    # implement this
    arr = np.array(arr)
    is_odd_arr = arr % 2
    odds = arr[is_odd_arr == 1]
```

```
    evens = arr[is_odd_arr == 0]
    new=np.concatenate((odds-1, evens*2))
    # print(new)
    return new.tolist()

assert solution([1, 2, 3, 4, 5]) == [0, 2, 4, 4, 8]
assert solution([2, 4]) == [4, 8]
assert solution([1, 3]) == [0, 2]
print("All tests passed")
```

```
[0 2 4 4 8]
[4 8]
[0 2]
All tests passed
```

▸ Solution

```
def solution(arr):
    evens = [x * 2 for x in arr if x % 2 == 0]
    odds = [x - 1 for x in arr if x % 2 != 0]
    return odds + evens
```

Problem 3 — String Substrings

Given a string `s`, split it into substrings of length `k`. For each substring, reverse it and capitalize the first character. Return the list of transformed substrings. If the last chunk is shorter than `k`, include it as-is (reversed and capitalized).

Example:

```
s = "abcdefgh", k = 3
chunks = ["abc", "def", "gh"]
output = ["Cba", "Fed", "Hg"]
```

In [96]:
```
n=3
```

Out[96]:
```
['abc', 'def', 'gh']
```

In [100…
```
s.replace
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[100], line 1
----> 1 s[1]='B'

TypeError: 'str' object does not support item assignment
```

In [107…
```
s.capitalize()
```

Out[107…
```
'Abcdefgh'
```

In [103…
```
for i in range(0,10,4):
    print(i)
```

```
0
4
8
```

In [108…
```
def solution(s, k):
    # implement this
    subs=[s[i:i+k] for i in range(0,len(s),k)]
    return [ sub[::-1].capitalize() for sub in subs]

assert solution("abcdefgh", 3) == ["Cba", "Fed", "Hg"]
assert solution("hello", 2) == ["Eh", "Ll", "O"]
print("All tests passed")
```

```
All tests passed
```

▸ Solution

```
def solution(s, k):
    result = []
    for i in range(0, len(s), k):
        chunk = s[i:i+k]
        reversed_chunk = chunk[::-1]
        result.append(reversed_chunk.capitalize())
    return result
```

Problem 4 — Number Digits

Given a list of integers, return a new list where each element is the sum of the digits of the corresponding integer. Negative numbers: use the absolute value.

Example:

```
nums = [123, -45, 0, 99]
output = [6, 9, 0, 18]
```

In [115…
```
s = str(abs(123))
sum(int(s[i:i+1]) for i in range(len(s)) )
```

Out[115… 6

In [116…
```
def solution(nums):
    # implement this
    result = []
    for num in nums:
        s = str(abs(num))
        result.append(
            sum(int(s[i:i+1]) for i in range(len(s)) ))
    return result
```

```
assert solution([123, -45, 0, 99]) == [6, 9, 0, 18]
assert solution([10, 100]) == [1, 1]
print("All tests passed")
```

```
All tests passed
```

▸ Solution

```
def solution(nums):
    return [sum(int(d) for d in str(abs(n))) for n in nums]
```

Problem 5 — Matrix Row/Column Max (the practice question)

This is the one from the CodeSignal practice environment.

Given an n×n matrix, return a list where result[i] = max of row i + max of column i.

In [110…

```
import numpy as np

def solution(matrix):
    np_matrix = np.array(matrix)
    # implement this (one line is fine)
    return (np.max(np_matrix, axis=0)
            + np.max(np_matrix, axis=1)).tolist()

assert solution([[2, 5, 8], [1, 2, 10], [-2, 0, -1]]) == [10, 15, 10]
print("All tests passed")
```

```
All tests passed
```

▸ Solution

```
def solution(matrix):
    np_matrix = np.array(matrix)
    return list(np_matrix.max(axis=1) + np_matrix.max(axis=0))
```